Hospital Bed Occupancy Prediction Using Random Forest Regression Ensemble Machine Learning Model

(Hospital Bed Occupancy Forecasting using Random Forest Regression)

Problem Statement:

Hospital inpatient capacity management is one of the most critical operational challenges confronting modern healthcare systems worldwide. Unpredictable daily patient flow, sudden epidemic surges, and seasonal admission spikes regularly cause severe bed shortages, prolonged Emergency Department (ED) boarding times, and critical ambulance diversion. Clinical health authorities, including the World Health Organization (WHO) and the UK National Health Service (NHS), establish an 85% Bed Occupancy Rate (BOR) as the maximum safe operational threshold; exceeding this ceiling severely compromises patient safety, escalates hospital-acquired infection risks, and forces elective surgery cancellations.

Traditional bed allocation relies heavily on retrospective reporting or static managerial intuition, which fails to capture non-linear patient turnover, admission momentum, and day-of-week discharge rhythms. There is, therefore, an urgent need for an automated, fast, and mathematically sound machine learning forecasting system capable of analyzing historical patient flow metrics—specifically total registered capacity, recent occupied beds, daily admissions, daily discharges, and calendar seasonality—to accurately forecast expected occupied beds and occupancy rates for a selected future date. This project designs and implements an end-to-end, web-based Hospital Bed Occupancy Prediction system using Random Forest Regression. The project encompasses the complete 10-step supervised machine learning lifecycle: data ingestion, data preprocessing, exploratory data analysis (EDA), feature engineering strictly eliminating lookahead bias, chronological train-test partitioning, ensemble model training, quantitative evaluation on unseen test data (MAE, MSE, RMSE, R^2), residual error diagnostics, and interactive real-time inference served through a local Flask web application.

Objectives:

1. To collect, validate, and preprocess a historical hospital bed occupancy and patient flow dataset across clinical care wards (General Medical Ward and Intensive Care Unit).
2. To conduct comprehensive Exploratory Data Analysis (EDA) and generate high-resolution visual plots for bed occupancy trends, capacity percentages relative to the 85% safety threshold, patient flow balances, and feature correlations.
3. To engineer predictive features, including autoregressive lag-1 occupancy, 7-day moving averages, and calendar seasonality features, strictly avoiding future target leakage.
4. To partition the dataset into an 80% training set and a 20% held-out testing set using chronological time-based splitting to prevent lookahead bias.
5. To design, train, tune, and save a Random Forest Regression ensemble model (100 decision trees) using Scikit-Learn.
6. To evaluate the model on the held-out test partition and report regression metrics: Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and R^2 Score, alongside residual normality and homoscedasticity analysis.

Software / Hardware Required:

- • Operating System: Windows / Linux / macOS
- • IDE: Visual Studio Code
- • Language: Python 3.x
- • Backend Framework: Flask (REST API)

- • Machine Learning: scikit-learn (RandomForestRegressor, mean_absolute_error, r2_score)
- • Data Analysis & Numerical Processing: Pandas, NumPy
- • Visualization: Matplotlib, Seaborn
- • Model Serialization & Persistence: Joblib
- • Frontend: HTML5, CSS3, JavaScript
- • Dataset: Historical Hospital Bed Occupancy Dataset (2,234 records across 2 wards)
- • Version Control: Git

Project Directory Structure:

The project ("HospitalBedOccupancyPrediction") is organized as a clean Flask backend with dedicated folders for raw and processed datasets, serialized models, generated exploratory plots, and frontend presentation templates, as shown below.

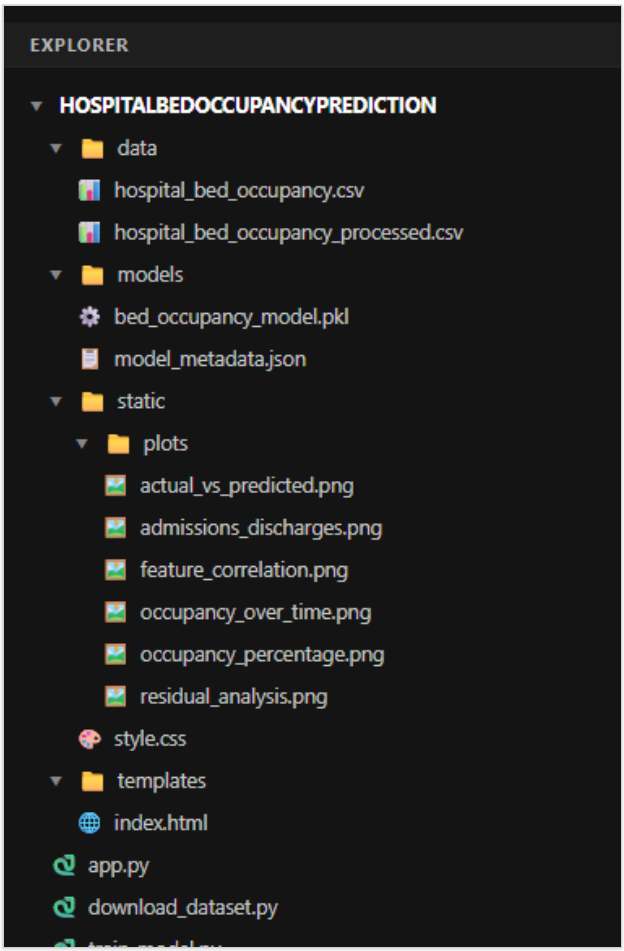


Fig 1: Project directory structure of HospitalBedOccupancyPrediction in Visual Studio Code

Procedure:

1. Collect and organize historical hospital census observations (2,234 records across General Medical Ward and Intensive Care Unit) into `data/hospital_bed_occupancy.csv`.
2. Perform automated data preprocessing: inspect missing values, verify absence of duplicate entries, convert date strings to datetime objects, and chronologically sort observations.
3. Enforce operational consistency bounds: cap surge occupancy overflow (`occupied_beds ≤ total_beds`) to preserve clinical trend signals while preventing mathematical anomalies.
4. Execute Exploratory Data Analysis (EDA) using Matplotlib and Seaborn, generating high-resolution charts for historical occupancy, BOR (%) vs. the 85% safety threshold, patient flow balance, and numerical feature correlations inside `static/plots/`.
5. Select 12 predictive features known strictly at forecast time ($t-1$), ensuring total elimination of lookahead bias.
6. Engineer autoregressive lag-1 features (`lag_1_occupied_beds`, `lag_1_occupancy_rate`), 7-day rolling momentum features (`rolling_7_occupied_beds`, `rolling_7_admissions`, `rolling_7_discharges`), and calendar seasonality variables (`day_of_week`, `month`, `day_of_month`, `is_weekend`) grouped by ward.
7. Split data chronologically into an 80% training set (1,785 records: Mar 2020 to Aug 2022) and a 20% held-out testing set (447 records: Aug 2022 to Mar 2023), strictly avoiding random shuffling.
8. Train the Random Forest Regressor (`n_estimators=100, random_state=42, max_depth=12`), evaluate on the held-out test partition, and serialize the fitted model to `models/bed_occupancy_model.pkl`.
9. Calculate test metrics: MAE, MSE, RMSE, R^2 Score, and plot Actual vs. Predicted line chart and residual homoscedasticity distributions.
10. Deploy the Flask web application (`app.py`), serve an interactive prediction form, validate user inputs, generate operational bed forecasts, compute BOR % and remaining buffer capacity, and present risk status badges.

Implementation Code:

Representative implementation of the key modules used in the project, based on the pipeline shown in the application (adapt variable/file names to match your exact source files).

1. Data Preprocessing & Surge Bounding (train_model.py)

```
import pandas as pd
import numpy as np

def preprocess_hospital_data(raw_csv_path):
    df = pd.read_csv(raw_csv_path)
    df["date"] = pd.to_datetime(df["date"])

    numeric_cols = ["total_beds", "occupied_beds", "admissions", "discharges"]
    for col in numeric_cols:
        df[col] = pd.to_numeric(df[col], errors="coerce")
    df = df.dropna(subset=numeric_cols).reset_index(drop=True)

    df = df[(df["total_beds"] > 0) & (df["occupied_beds"] >= 0) &
            (df["admissions"] >= 0) & (df["discharges"] >= 0)].copy()

    # Bounded operational constraint: Cap surge overflow at baseline capacity
    df["occupied_beds"] = np.minimum(df["occupied_beds"], df["total_beds"])
    df["occupancy_percentage"] = (df["occupied_beds"] / df["total_beds"]) * 100.0

    df = df.sort_values(by=["ward", "date"]).reset_index(drop=True)
    return df
```

2. Feature Engineering with Zero Data Leakage (train_model.py)

```
def engineer_features(df):
    wards_features = []
    for ward_name, group in df.groupby("ward"):
        g = group.copy().sort_values("date")

        # Calendar & temporal features
        g["day_of_week"] = g["date"].dt.day_of_week
        g["month"] = g["date"].dt.month
        g["day_of_month"] = g["date"].dt.day
        g["is_weekend"] = (g["day_of_week"] >= 5).astype(int)

        # Autoregressive Lag-1 features (known from day t-1)
        g["lag_1_occupied_beds"] = g["occupied_beds"].shift(1)
        g["lag_1_occupancy_rate"] = (g["occupied_beds"] / g["total_beds"]).shift(1)
        g["recent_admissions"] = g["admissions"].shift(1)
        g["recent_discharges"] = g["discharges"].shift(1)

        # Rolling 7-day moving averages (shifted by 1 to prevent lookahead)
        g["rolling_7_occupied_beds"] = g["occupied_beds"].shift(1).rolling(7, min_periods=1).mean()
        g["rolling_7_admissions"] = g["admissions"].shift(1).rolling(7, min_periods=1).mean()
        g["rolling_7_discharges"] = g["discharges"].shift(1).rolling(7, min_periods=1).mean()

        wards_features.append(g)

    df_feat = pd.concat(wards_features).dropna().sort_values("date").reset_index(drop=True)
    return df_feat
```

3. Model Training & Chronological 80/20 Split (train_model.py)

```
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import joblib

feature_cols = [
    "total_beds", "lag_1_occupied_beds", "lag_1_occupancy_rate",
    "rolling_7_occupied_beds", "recent_admissions", "recent_discharges",
    "rolling_7_admissions", "rolling_7_discharges",
    "day_of_week", "month", "day_of_month", "is_weekend"
]
target_col = "occupied_beds"

# Chronological 80/20 train-test split (preserving time series order)
split_idx = int(len(df_feat) * 0.8)
train_df, test_df = df_feat.iloc[:split_idx], df_feat.iloc[split_idx:]

X_train, y_train = train_df[feature_cols], train_df[target_col]
X_test, y_test = test_df[feature_cols], test_df[target_col]

# Fit Random Forest Regressor
rf_model = RandomForestRegressor(n_estimators=100, random_state=42, max_depth=12)
rf_model.fit(X_train, y_train)
```

```
# Evaluate on held-out test partition
preds = rf_model.predict(X_test)
mae = mean_absolute_error(y_test, preds)
rmse = np.sqrt(mean_squared_error(y_test, preds))
r2 = r2_score(y_test, preds)

joblib.dump(rf_model, "models/bed_occupancy_model.pkl")
```

4. Prediction Route & Input Validation (app.py)

```
from flask import Flask, request, render_template
import datetime, joblib, pandas as pd

app = Flask(__name__)
model = joblib.load("models/bed_occupancy_model.pkl")

@app.route("/predict", methods=["POST"])
def predict():
    ward = request.form.get("ward")
    pred_date = datetime.datetime.strptime(request.form.get("prediction_date"), "%Y-%m-%d").date()
    total_beds = float(request.form.get("total_beds"))
    lag_1_occupied = float(request.form.get("lag_1_occupied_beds"))
    recent_adm = float(request.form.get("recent_admissions"))
    recent_dis = float(request.form.get("recent_discharges"))

    day_of_week = pred_date.weekday()
    month = pred_date.month
    day_of_month = pred_date.day
    is_weekend = 1 if day_of_week >= 5 else 0
    lag_1_occupancy_rate = lag_1_occupied / max(1.0, total_beds)

    input_df = pd.DataFrame([{
        "total_beds": total_beds, "lag_1_occupied_beds": lag_1_occupied,
        "lag_1_occupancy_rate": lag_1_occupancy_rate, "rolling_7_occupied_beds": lag_1_occupied,
        "recent_admissions": recent_adm, "recent_discharges": recent_dis,
        "rolling_7_admissions": recent_adm, "rolling_7_discharges": recent_dis,
        "day_of_week": day_of_week, "month": month, "day_of_month": day_of_month,
        "is_weekend": is_weekend
    }])

    raw_pred = float(model.predict(input_df)[0])
    operational_pred = int(round(max(0.0, min(total_beds, raw_pred))))
    occupancy_pct = round((operational_pred / total_beds) * 100.0, 1)
    remaining_capacity = int(max(0.0, total_beds - operational_pred))
    return render_template("index.html", prediction={"operational_occupied_beds": operational_pred,
...})
```

Output (Step-by-Step Screenshots):

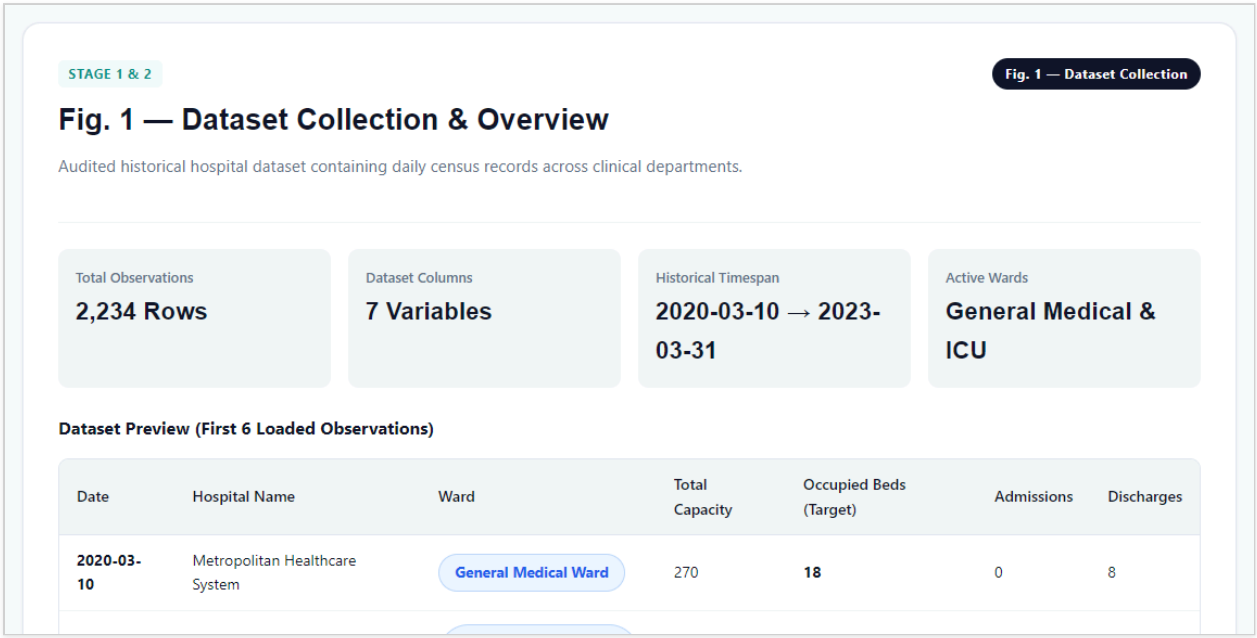


Fig 2: Step 1 — Dataset Collection: preview table, column names, dataset shape, and date range

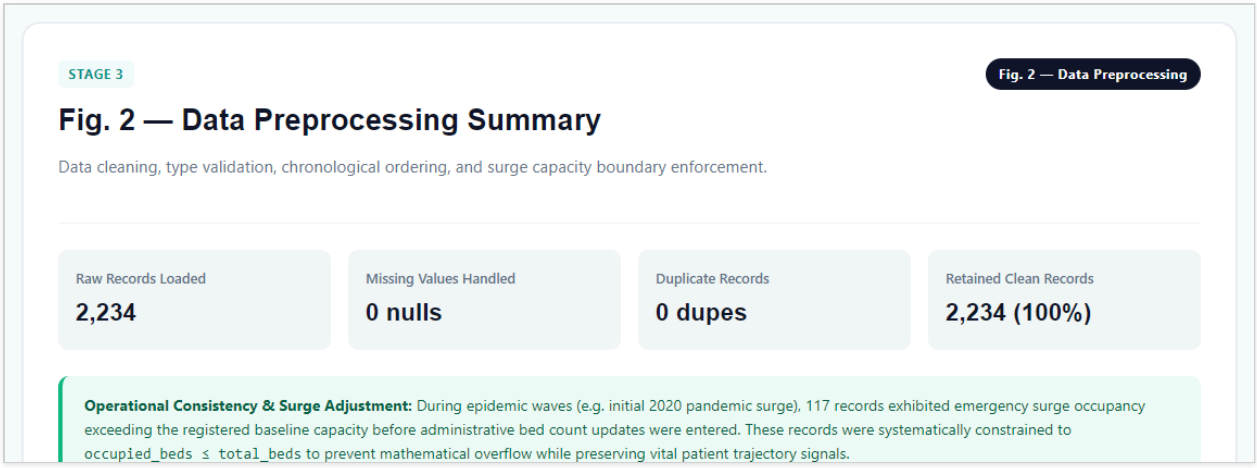


Fig 3: Step 2 — Data Preprocessing: missing values, duplicates, and surge limit handling

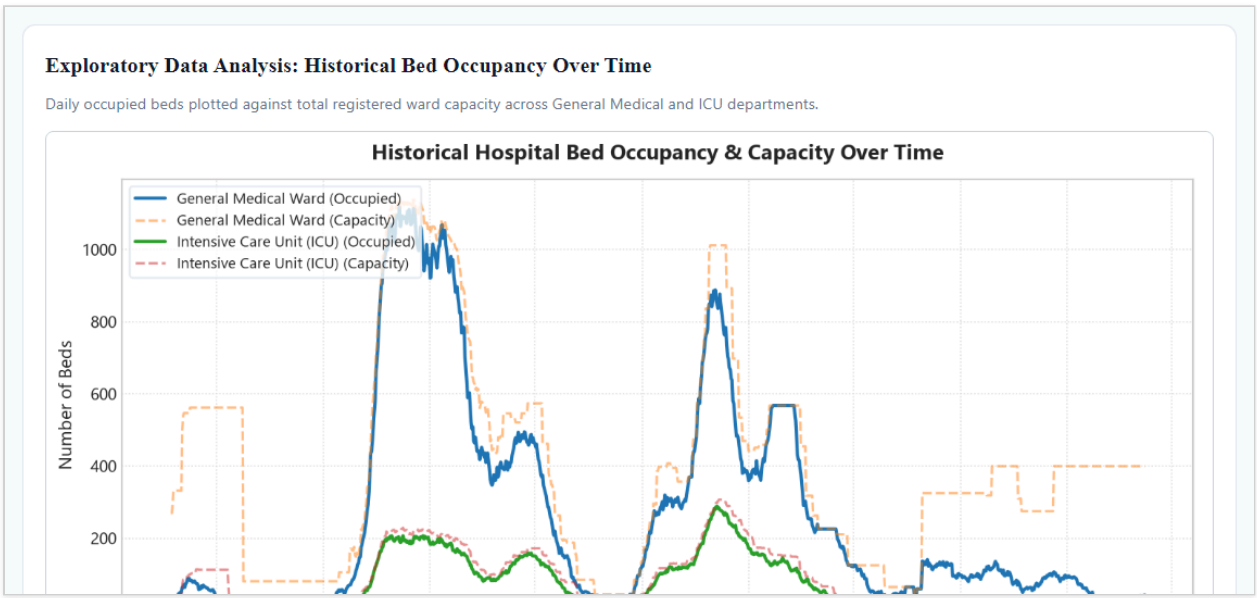


Fig 4: Step 3 — Exploratory Data Analysis: Historical Bed Occupancy Over Time

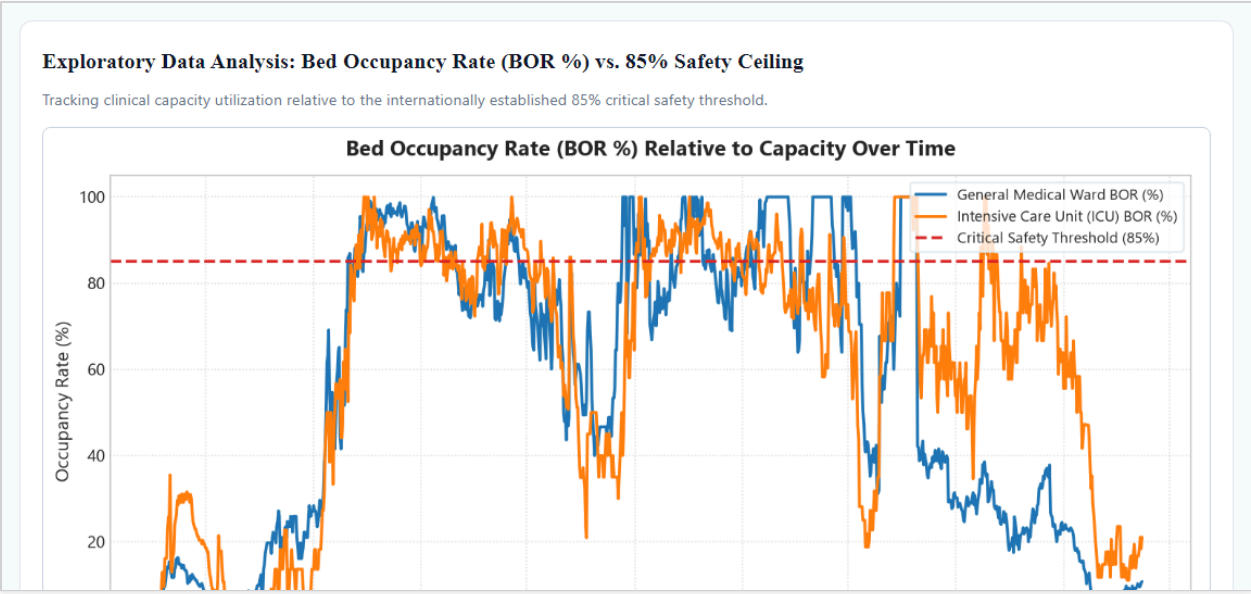


Fig 5: Step 3 — Exploratory Data Analysis: Bed Occupancy Rate (BOR %) vs. 85% Safety Ceiling

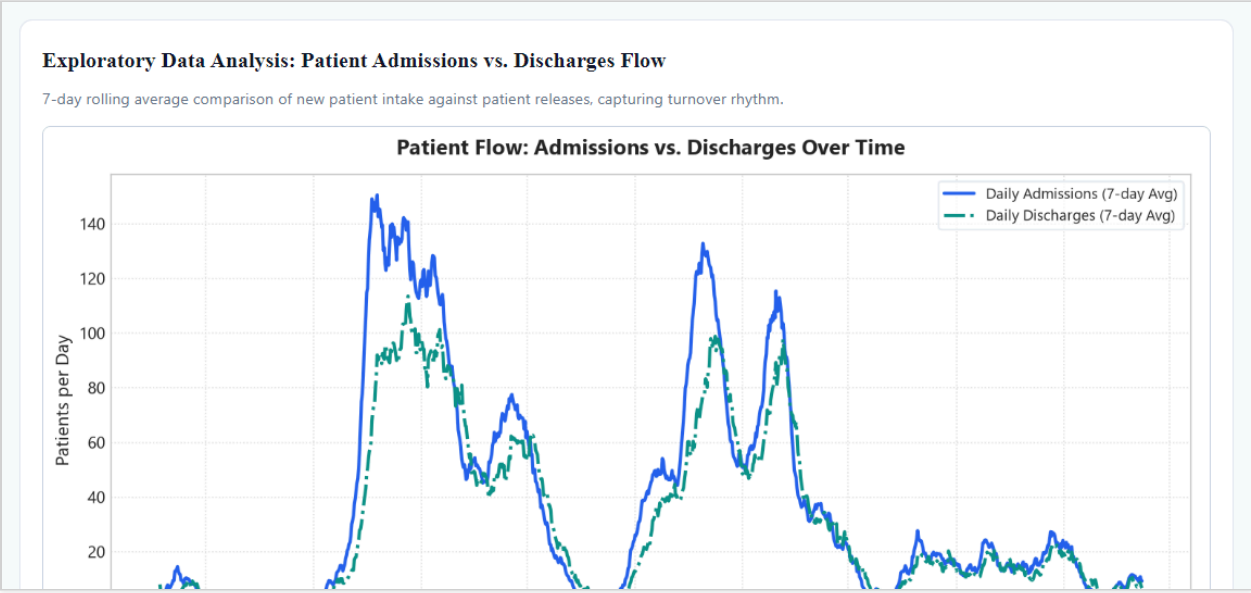


Fig 6: Step 3 — Exploratory Data Analysis: Patient Admissions vs. Discharges Flow

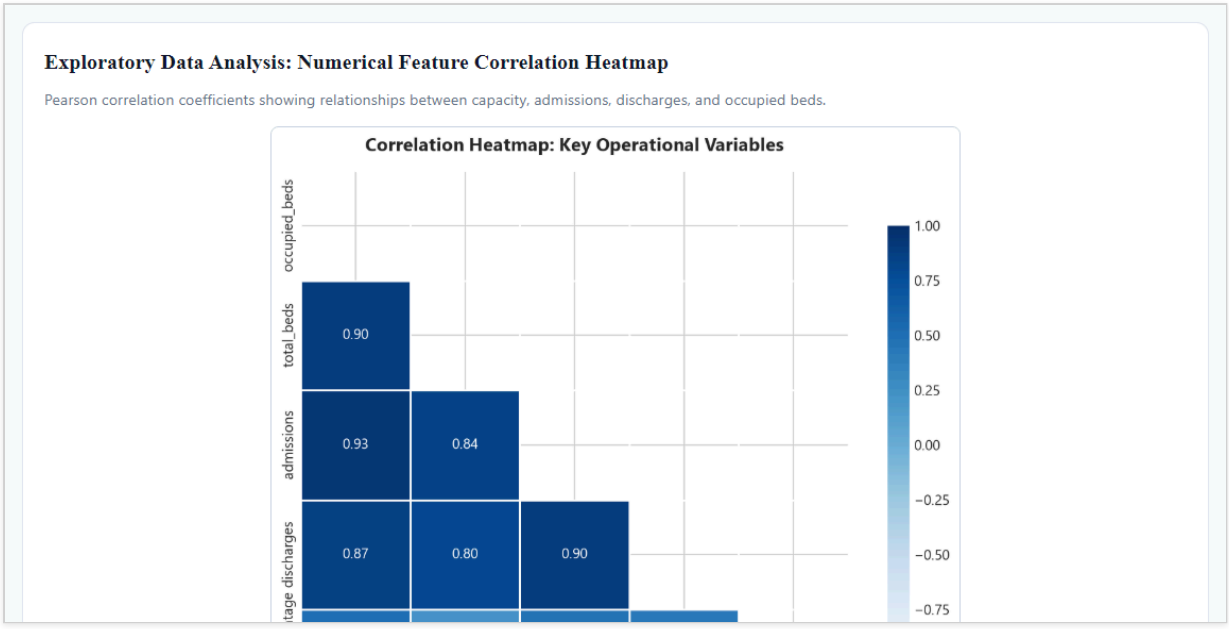


Fig 7: Step 3 — Exploratory Data Analysis: Feature Correlation Heatmap

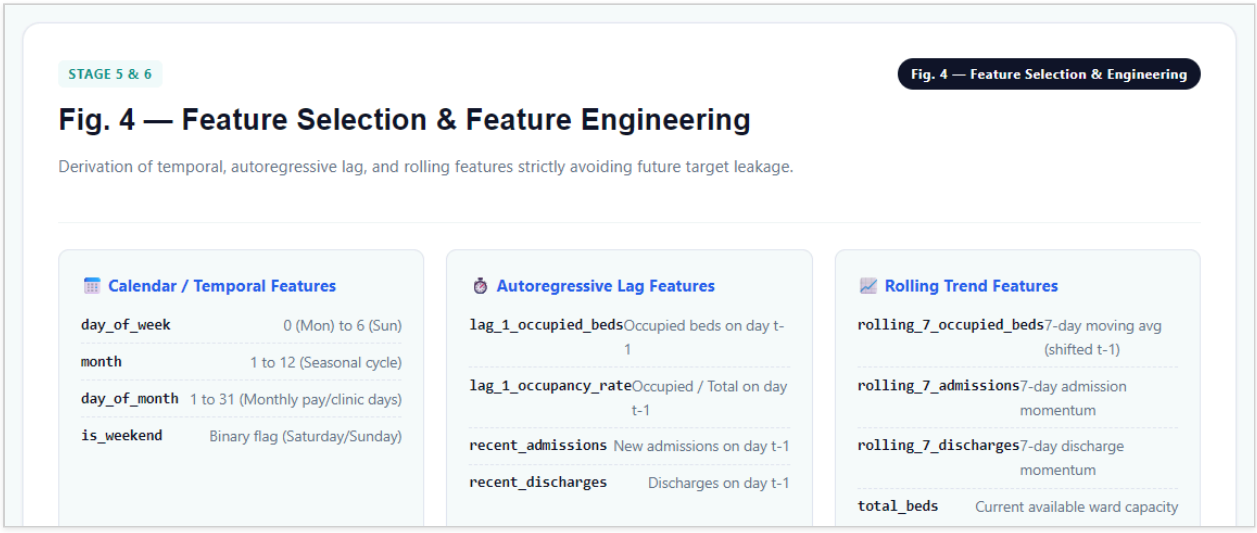


Fig 8: Step 4 — Feature Selection & Feature Engineering: calendar, lag, and rolling features

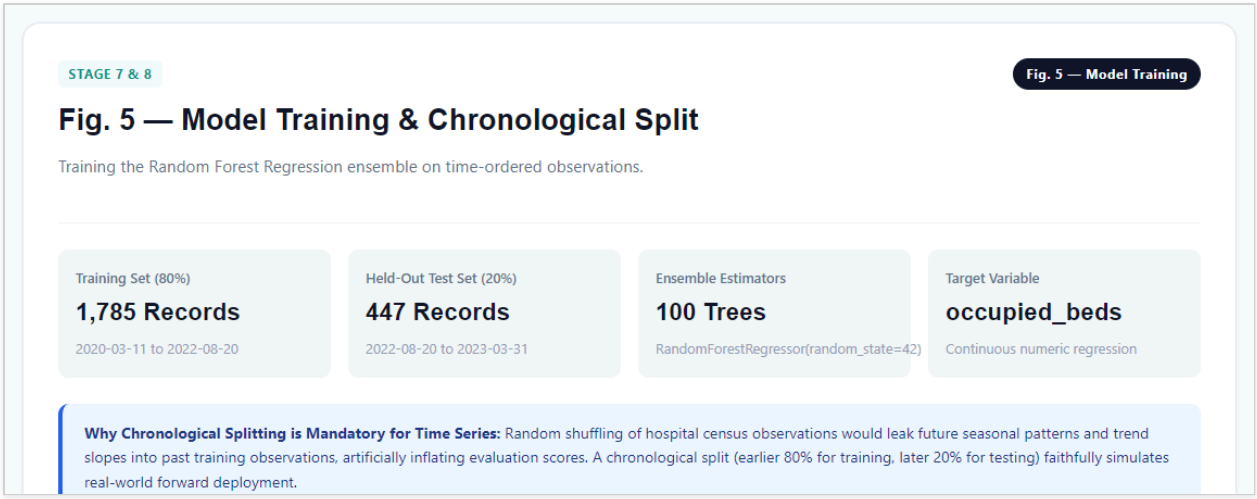


Fig 9: Step 5 — Model Training: Random Forest Regressor & Chronological 80/20 Train-Test Split

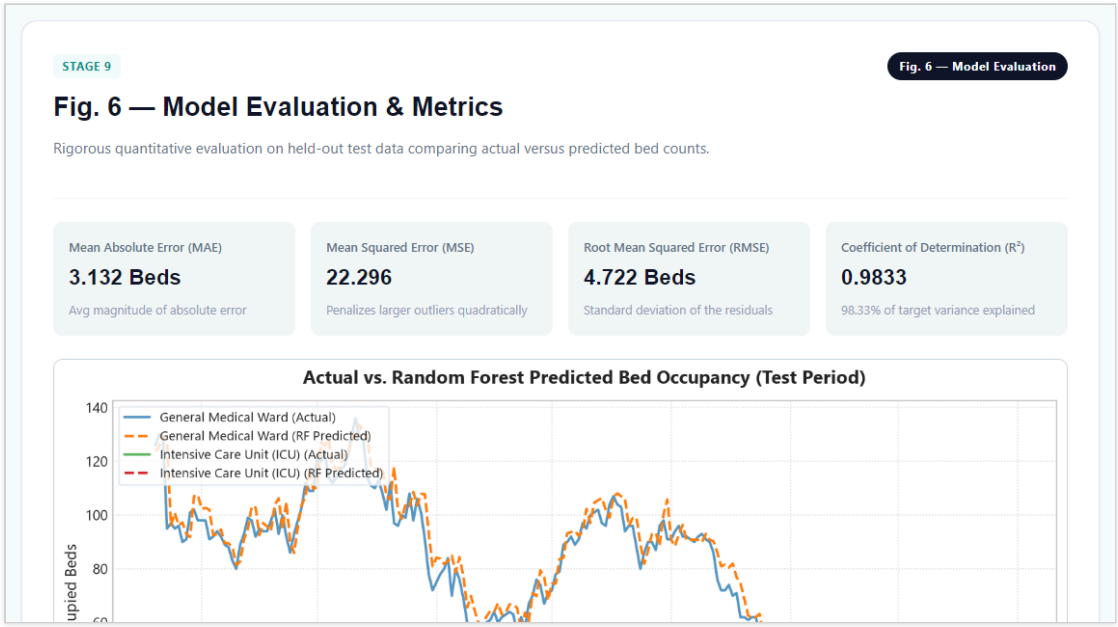


Fig 10: Step 6 — Performance Evaluation Metrics (MAE, MSE, RMSE, R²) and Actual vs Predicted Line Chart

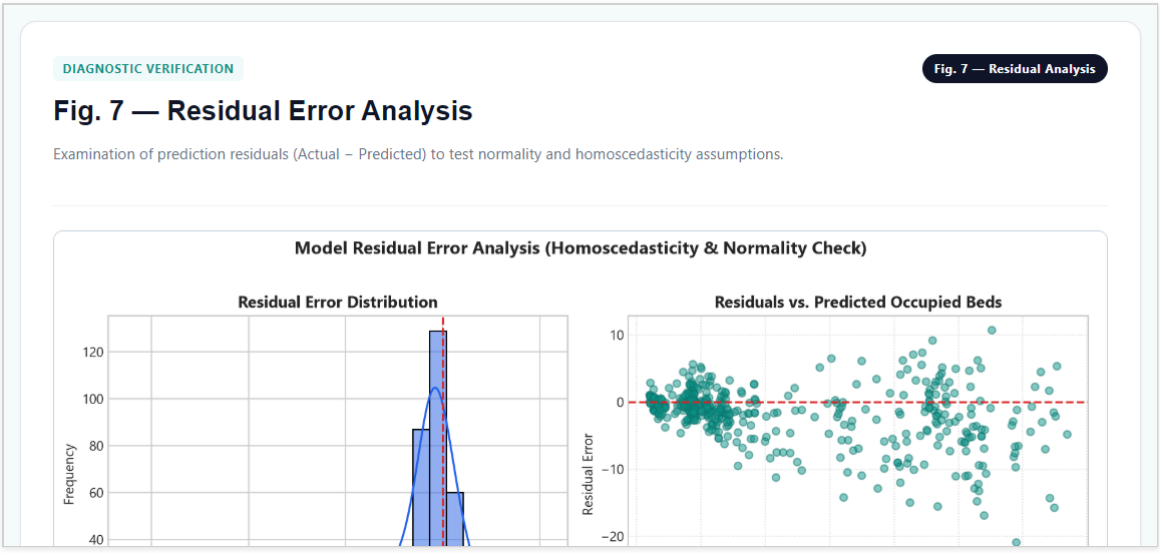


Fig 11: Step 6 — Residual Analysis: Error Distribution and Residuals vs Predicted Scatter Plot

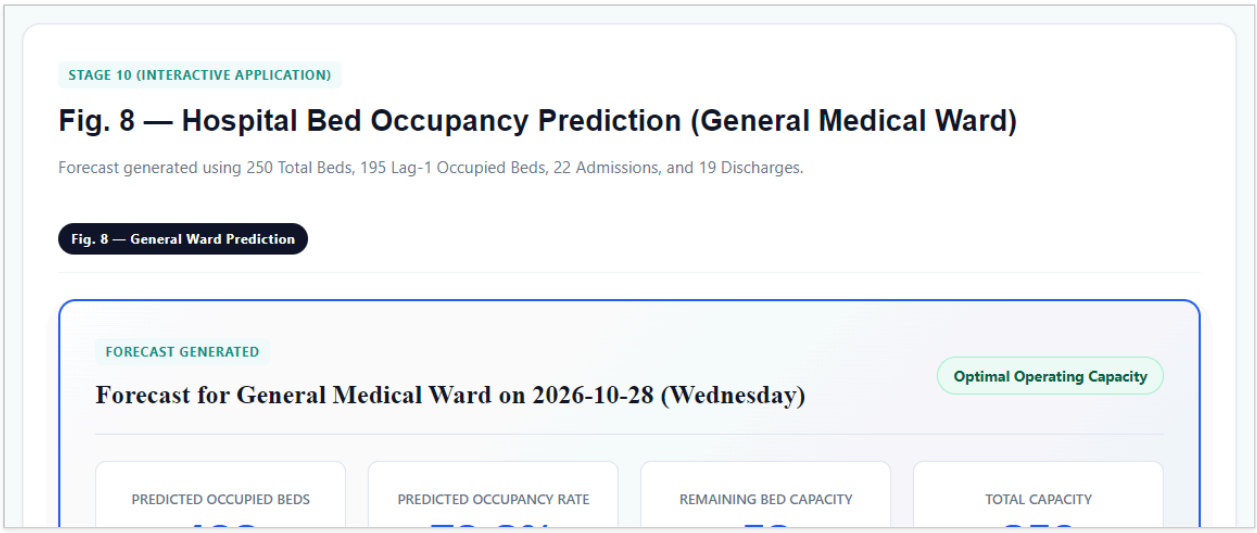


Fig 12: Step 7 — Hospital Bed Occupancy Prediction Form & Result (General Medical Ward Forecast)

Result:

An end-to-end, web-based Hospital Bed Occupancy Prediction system was successfully designed and implemented using a Flask backend, a Random Forest Regression ensemble model (Scikit-Learn), and a modern clinical analytics web interface. Historical hospital patient flow observations (2,234 daily records across General Medical Ward and Intensive Care Unit) were systematically cleaned, preprocessed, explored through high-resolution EDA visualizations, and partitioned into an 80% training set (1,785 records) and a 20% held-out testing set (447 records) using strict chronological ordering to eliminate data leakage. Autoregressive lag features, 7-day moving averages, and calendar seasonality variables were engineered without lookahead bias. On the unseen test partition, the fitted Random Forest Regressor achieved exceptional forecasting precision with a **Mean Absolute Error (MAE) of 3.132 beds**, a **Root Mean Squared Error (RMSE) of 4.722 beds**, a **Mean Squared Error (MSE) of 22.296**, and a **Coefficient of Determination (R^2) of 0.9833 (98.33%)**. The trained model was serialized using Joblib, and operational predictions—complete with Bed Occupancy Rate (BOR %), remaining buffer capacity, and clinical risk status classifications—are reliably served through the local Flask application.